

## Information Security

## Assignment #01

Date: 01-09-2026

Course code: CS3002

Instructor(s)

Total marks: 100

Aqsa Khalid

Questions: 1

---

Roll No

---

Section

---

Student Signature

---

### What You Will Build (and Break)

Earlier you built tools that inspect a system from the outside. In this assignment you step *inside* a real web application and attack it the way a professional penetration tester does on their first day on the job, then you switch sides and fix the very flaw you just exploited.

You are given a deliberately vulnerable web application called OWASP Juice Shop. It looks like a normal online store (products, a cart, user accounts, an admin panel) but underneath it is full of real, industry-relevant security flaws. For each flaw you will: (1) exploit it, (2) find the vulnerable line of code that caused it, and (3) choose the correct fix. Attacker *and* defender, in one exercise.

This is the single most employable skill in entry-level information security: web application penetration testing.

#### About OWASP Juice Shop

Juice Shop is an open-source project maintained by the OWASP Foundation, purpose-built for security training. It contains 100+ challenges covering the OWASP Top 10 (the industry's standard list of the most critical web risks) and tracks your progress on a built-in Score Board. Crucially, many challenges (including all seven in this assignment) come with a built-in Coding Challenge. After you exploit a vulnerability, you can click "Jump to related coding challenge" and the app shows you its own real source code and asks you to:

- Find It: click the exact line(s) of code that contain the vulnerability.
- Fix It: choose the correctly patched version of that code.

The app checks your answers automatically and marks them green when correct. This is the defender's half of the job, and it is a required part of every challenge below.

---

### Scope and Ethics (Read This First)

- You may only attack the OWASP Juice Shop instance you run yourself.
  - You may not point any tool or technique from this assignment at any website, server, or network you do not own — not the university's, not a company's, not a friend's. This is illegal under Pakistan's Prevention of Electronic Crimes Act (PECA) 2016 and equivalent laws worldwide, and carries real criminal penalties.
  - Everything you need is inside the sandbox. There is no reason to touch anything else.
- Breaching scope is treated as disciplinary misconduct, separate from the plagiarism policy.

### Part 0 — Environment Setup (5 marks)

# National University of Computer and Emerging Sciences

## Lahore Campus

### 1. Check your Node.js version

Juice Shop needs Node.js version 22 or 24. An older version (18 or below) will cause the install to fail with confusing errors. In a terminal, run:

```
node -v
```

If it prints v22.x or v24.x → you're good. If it prints anything older (e.g. v18) → install Node 22 or 24 first.

### 2. Run the application (pick ONE method)

**Do not use the public online demo. Run your own local copy:**

- Method A: Packaged download (fastest, no build step). Go to the Juice Shop GitHub Releases page, download `juice-shop-<version>_node24_<os>_x64.zip` for your OS, unzip it, open a terminal in that folder, run `npm start`, and open <http://localhost:3000>.
- Method B: From source with Node.js.

```
git clone https://github.com/juice-shop/juice-shop.git --depth 1
```

```
cd juice-shop
```

```
npm install # slow — several minutes, this is normal; wait for it to finish
```

```
npm start
```

Then open <http://localhost:3000>. (`npm install` builds the whole app, so it is slow. Slow is normal — let it complete.)

- Method C: Docker (if you already use it).

```
docker run --rm -p 127.0.0.1:3000:3000 bkimminich/juice-shop
```

Then open <http://localhost:3000>.

After it loads

- If your editor asks to "Reopen in Container," dismiss it — you don't need that.
- Turn hints on: find the Score Board (that's Challenge 1), and use the toggle to enable challenge hints. The in-app hints are allowed and encouraged.

**Deliverable for Part 0: a screenshot of your running Juice Shop homepage, with the terminal that launched it visible in the same shot.**

---

## Background — What These Vulnerabilities Are (Read This Before You Start)

1. **SQL Injection.** Websites store data in a database and fetch it using a query language called SQL. A login builds a query like *"find the user whose email is [what you typed]."* If the site pastes your text straight into that query, you can type text that is actually SQL *code* instead of an email — and the database runs it, letting you (for example) log in without a real password. Root problem: the app treated your input as code when it should have treated it as data.

# National University of Computer and Emerging Sciences

## Lahore Campus

2. **Cross-Site Scripting (XSS)**. Browsers run JavaScript. If a site takes something you type and puts it back on the page without cleaning it, and your text contains JavaScript, the browser *runs* it. Root problem: the app showed user input as if it were part of its own page.

3. **Broken Access Control**. "Authentication" = proving *who you are* (logging in). "Authorisation" = checking *what you're allowed to do*. Broken access control is checking the first but forgetting the second — so a normal user reaches an admin-only page just by knowing its address. Root problem: the app assumed you wouldn't try instead of actively blocking you.

4. **Authentication Weaknesses**. Flaws in the login/identity system itself — a weak guessable password, or a "forgot password" security question whose answer is public. Root problem: the identity checks are too easy to defeat.

**Naming which of these four you exploited is half of every explanation.**

---

## The Challenges (7 total)

Find each challenge by its exact name on the Score Board (given in bold below). For every challenge **you must do all three parts: exploit it, complete its Find It coding challenge, and complete its Fix It coding challenge**. Then **document it** using the **Reporting Template**. The objectives tell you *what* to achieve, not *how*.

### Tier 1: Recon & Basics (2x6 = 12 marks)

1. **"Score Board"**: Find the carefully hidden Score Board page. *Hint: the front-end code your browser downloaded already knows this route.*
2. **"DOM XSS"**: Make the application run a JavaScript pop-up in your browser via a crafted input. *Category: XSS. Hint: try the site's search field.*
3. **"Confidential Document"**: Access a document on the server that customers were never meant to see. *Category: Sensitive Data Exposure. Hint: follow a link the site exposes by accident, then look at the folder it sits in.*

### Tier 2: Authentication & Access (3x9 = 27 marks)

4. **"Login Admin"**: Log in as the administrator without knowing the admin password. *Category: Injection. Hint: think about what the login form does with your email text before it reaches the database.*
5. **"Password Strength"**: Log in as the administrator without using SQL injection — by guessing the weak password. *Category: Broken Authentication. Hint: the admin never changed a very common default.*
6. **"Admin Section"**: Reach the hidden administration area of the store. *Category: Broken Access Control. Hint: the route exists in the front-end code, but this one also checks your privileges.*

### Tier 3: Exploitation (12 marks)

7. **"Reset Jim's Password"**: Take over the user "Jim" by defeating his "forgot password" security question. *Category: Broken Authentication / OSINT. Hint: Jim's identity is a well-known pop-culture reference; his security answer can be found from public information about that character.*

## Reporting Template (use for every challenge)

For each of the seven challenges, your report must contain:

1. Challenge name and OWASP category.
2. **How you exploited it:** numbered steps and the exact input/payload you used.
3. **Screenshot A:** exploit succeeded (the green "challenge solved" banner on your instance).
4. **Screenshot B:** "Find It" solved (the vulnerable line correctly identified, marked green).
5. **Screenshot C:** "Fix It" solved (the correct patch chosen, marked green).
6. **In 2–3 sentences:** the root cause and why the fix works, what mistake caused the flaw, and why the patch you selected stops it.

The three screenshots prove you did the work; the 2–3 sentences prove you understood it. Both are needed for full marks.

---

## If You Get Stuck (This Is Expected)

Nobody solves these cold. You have three levels of help, and using them is allowed:

1. The hints in this document (under each challenge).
2. Juice Shop's built-in hint system - each challenge on the Score Board has a hint button (often a second, more detailed one too). Turn hints on.

Using hints to *learn* is fine. Copying a walkthrough's text into your report is not.